

The MAYA Shuffle Argument

Verifier Specification

(Draft Version)

MAYA is a logarithmic-size, zero-knowledge shuffle argument with a transparent setup for mixnet-based electronic voting. It uses the Fiat-Shamir heuristic to produce a non-interactive and universally verifiable proof that a set of randomizable ciphertexts has been correctly shuffled (re-encrypted and permuted).

This document describes the data formats, Fiat-Shamir instantiation, and verification procedure of the MAYA implementation. It is intended to support implementors of independent verifiers in processing these proofs.

Status: This document is a working draft provided as supplementary material. Some details may be revised as the implementation evolves.

Contents

1	Notation and Cryptographic Setting	3
2	Group Arithmetic and Encodings	3
2.1	Group and Field Parameters	3
2.2	Deserializing Points and Scalars from Files	3
2.3	Hash-to-Group Function	3
3	Public Keys and Generator Setup	4
3.1	Public Keys	4
3.2	Bulletproof Generator Vectors	4
4	Fiat-Shamir Transcript	4
4.1	Transcript Concept	4
4.2	Concrete Instantiation	5
5	Verifier Inputs and Outputs	5
5.1	Padding Parameters	5
5.2	Input Ciphertext File	6
5.3	Output Bundle File	6
5.4	R1CSProof Wire Format	6
6	Verification Steps	7
6.1	Read Inputs and Parse Proof	7
6.2	Derive Challenge Vector $\mathbf{e}$	7
6.3	Compute Aggregated Input Ciphertexts	7
6.4	Replay the Proof Transcript	8
6.5	Compute Constraint Weight Vectors	8
6.6	IPA Verification Scalars	9
6.7	eCP Verification Scalars	9
6.8	Assemble Scalar Coefficients	9
6.9	Final MSM	9
A	Verification Algorithms	11
A.1	Algorithm: Compute Weight Vectors	11
A.2	Algorithm: IPA Verification Scalars	11
A.3	Algorithm: eCP Verification Scalars	13
B	Notation Correspondence	15

1 Notation and Cryptographic Setting

Let $(\mathbb{G}, p, g)$ be a cyclic group of prime order p with generator g , written in additive notation. All scalar arithmetic is in $\mathbb{Z}_p$.

x, α, u	Scalars (lowercase italic/Greek), elements of $\mathbb{Z}_p$.
A, V, T	Points (uppercase italic), elements of $\mathbb{G}$.
$\mathbf{a}, \mathbf{g}$	Vectors (bold).
$\langle \mathbf{a}, \mathbf{b} \rangle$	Inner product $\sum_i a_i b_i \in \mathbb{Z}_p$.
$\text{MSM}(\mathbf{G}, \mathbf{s}, w)$	Multi-scalar multiplication of width w : $\sum_{i=1}^w s_i \cdot G_i$.
$\mathcal{O}$	Identity element (point at infinity).
$\text{Compress}(P)$	The canonical byte encoding of point P (32 bytes for Ristretto255).

2 Group Arithmetic and Encodings

2.1 Group and Field Parameters

MAYA operates over a cyclic group $\mathbb{G}$ of prime order p with generator g , written in additive notation ($a + b$ for group addition, $s \cdot P$ for scalar multiplication). Scalar arithmetic is in $\mathbb{Z}_p$.

Ristretto255 instantiation.

- *Group:* Ristretto255 [3], a prime-order group constructed over Curve25519. $p = 2^{252} + 2774231777372353535851937790883648493$.
- *Point encoding:* Each point is encoded as 32 bytes using the Ristretto255 compression map (RFC 9496 [4]). Decompression either succeeds (yielding a valid group element) or fails (the point is invalid and the proof is rejected).
- *Scalar encoding:* A scalar $s \in \mathbb{Z}_p$ is encoded as 32 bytes in little-endian byte order. A scalar is *canonical* if the 32-byte value, interpreted as an unsigned integer, is in $[0, p-1]$. Reject non-canonical scalars.
- *Identity element:* The identity $\mathcal{O}$ is encoded as 32 zero bytes.

Other curves. For a different group (e.g. P-256): replace the point encoding/decoding, the scalar encoding, and the hash-to-group function (§2.3). All protocol logic remains identical.

2.2 Deserializing Points and Scalars from Files

Given a byte array b of length 32:

- To obtain a point $P \in \mathbb{G}$: Apply the Ristretto255 decompression map to b . If decompression fails, REJECT the proof.
- To obtain a scalar $s \in \mathbb{Z}_p$: Interpret b as a 256-bit unsigned integer in little-endian byte order. If the integer is $\geq p$, REJECT.

2.3 Hash-to-Group Function

$\text{HashToGroup}(m)$: compute $h \leftarrow \text{SHA3-512}(m)$ (64 bytes), then apply $\text{MapToGroup}(h)$.

For Ristretto255, MapToGroup is the `from_uniform_bytes` function (RFC 9496 [4], §4.3.4): interprets the 64 bytes as two 255-bit field elements, maps each to a point on Curve25519 via the Elligator 2 map, and adds the two resulting points. This produces a uniformly distributed group element. For other curves, use the hash-to-curve standard (RFC 9380 [5]).

3 Public Keys and Generator Setup

3.1 Public Keys

The verifier reads the public key (g, f, h) from the input ciphertext file (§5.2). These were published on the election bulletin board by the election authority before the shuffle.

- g , the ElGamal generator, part of the election public key.
- f , the ElGamal encryption key ($f = g^{\text{sk}}$ where sk is the threshold-shared secret key held by election trustees).
- h , the Pedersen blinding base, used for commitments.

The blinding base h is derived deterministically from g : $h = \text{HashToGroup}(\text{Compress}(g))$. An independent verifier may recompute h from g and check that it matches the value read from the file. This is an optional integrity check, not a required verification step.

3.2 Bulletproof Generator Vectors

The generator vectors $\mathbf{g} = (g_1, \dots, g_N)$ and $\mathbf{h} = (h_1, \dots, h_N)$ are deterministic: both prover and verifier derive them from the same algorithm.

Algorithm: $\text{GeneratorChain}(\text{label}) \rightarrow \text{sequence of points in } \mathbb{G}$

1. Initialize a SHAKE256 instance.
2. Absorb the 15-byte ASCII string "GeneratorsChain".
3. Absorb the byte array label .
4. Finalize into XOF (extensible output) mode.
5. To produce each generator: read 64 bytes from the XOF stream and apply MapToGroup . Repeat sequentially for as many generators as needed.

Generator capacity N . The verifier must generate at least N generators in each vector, where N is the padded number of committed variables (read from the proof, §5.1). In the current version of MAYA, there is only 1 party, so the generator vectors are $\mathbf{g} = \mathbf{G_vec}[0][0 \dots N-1]$ and $\mathbf{h} = \mathbf{H_vec}[0][0 \dots N-1]$.

4 Fiat–Shamir Transcript

The MAYA proof is made non-interactive via the Fiat–Shamir transform [2].

4.1 Transcript Concept

The transcript is a running hash state that supports two operations:

1. $\text{Absorb}(l, d)$: feed a label l (byte string) and data d (byte array) into the hash state.
2. $\text{Squeeze}(l) \rightarrow s$: produce a challenge scalar $s \in \mathbb{Z}_p$ from the current hash state, using label l .

The challenges depend on *all* previously absorbed data. Any change to any absorbed value produces different challenges.

4.2 Concrete Instantiation

The MAYA implementation uses the Merlin transcript library [8], which is built on the STROBE protocol framework [9] (based on the Keccak permutation [6]).

The Merlin library is available in Rust, C, Python, and Go. An independent verifier for the current MAYA-Ristretto must use Merlin (or a compatible re-implementation of the STROBE-based protocol) to produce matching challenges.

Operation	Description
<code>Transcript::new(l)</code>	Create a new transcript with protocol label l (byte string).
<code>append_message(l, d)</code>	Absorb byte array d under label l .
<code>challenge_bytes(l, n)</code>	Squeeze n bytes of output under label l . Returns an n -byte array.
<code>challenge_scalar(l)</code>	Squeeze 64 bytes under label l , interpret them as a 512-bit unsigned integer (little-endian byte order), and reduce modulo p to obtain a scalar $\in \mathbb{Z}_p$. The wide (64-byte) reduction ensures negligible statistical bias ($< 2^{-253}$).

Convenience wrappers used in this spec:

- Absorb point P : `append_message( $l$ , Compress( $P$ ))` (32 bytes).
- Absorb scalar s : `append_message( $l$ ,  $s$  as 32 bytes LE)`.
- Absorb integer n : `append_message( $l$ ,  $n$  as 8 bytes LE)`.

5 Verifier Inputs and Outputs

5.1 Padding Parameters

Symbol	Type	Description
n	$\mathbb{Z}_{>0}$	Number of real ciphertexts.
k	$\mathbb{Z}_{\geq 2}$	Folding factor (recommended: $k = 4$).
d	$\mathbb{Z}_{\geq 1}$	Recursion depth = $\lceil \log_k n \rceil$.
m	$\mathbb{Z}_{\geq 1}$	Base-case vector size after d rounds.
N	$\mathbb{Z}_{\geq n}$	Padded size.

Iterative padding. Given $N_0 = N$ and k :

```

for  $r = 1$  to  $d$  do
   $\text{rem} \leftarrow N_{r-1} \bmod k$ ;    $\text{pad} \leftarrow (k - \text{rem}) \bmod k$ 
   $N_r \leftarrow (N_{r-1} + \text{pad})/k$ 
end for

```

The base-case size is $m = N_d$. The values k, d, m are read from the proof (IPA sub-proof header). The verifier computes the round-length sequence $(N_0, \dots, N_d)$ and checks $N_d = m$.

To verify a MAYA shuffle proof, the verifier needs two files; one is produced by the mix server (the prover) and one is published on the public board:

File	Contents
<code>maya_ristretto_bench_input_ciph.bin</code>	Input ciphertexts and public key (g, f, h) .
<code>maya_ristretto_bench_output_bundle.bin</code>	Shuffled ciphertexts, proof, and commitment V .

The verifier reads both files, parses the proof, regenerates the Bulletproof generators (§3.2), and runs the verification algorithm (§6). The output is a single decision ACCEPT or REJECT.

5.2 Input Ciphertext File

Offset	Size	Field
0	8	u64 LE: n (number of ciphertexts).
8	$n \times 32$	$C_{1,1}, \dots, C_{n,1}$: first ciphertext components (compressed points).
$8 + 32n$	$n \times 32$	$C_{1,2}, \dots, C_{n,2}$: second ciphertext components.
$8 + 64n$	32	g : ElGamal generator.
$8 + 64n + 32$	32	f : ElGamal public key.
$8 + 64n + 64$	32	h : Pedersen blinding base.

5.3 Output Bundle File

Offset	Size	Field
0	8	u64 LE: bincode proof length ($= L$).
8	L	Bincode-serialized R1CSProof (see §5.4).
$8 + L$	32	V : permutation commitment (compressed point).
$8 + L + 32$	8	u64 LE: n (C1' count).
$8 + L + 40$	$32n$	$C'_{1,1}, \dots, C'_{n,1}$: shuffled first components.
$8 + L + 40 + 32n$	8	u64 LE: n (C2' count).
$8 + L + 48 + 32n$	$32n$	$C'_{1,2}, \dots, C'_{n,2}$: shuffled second components.

The R1CSProof is serialized via `bincode::serialize`, which wraps the output of `R1CSProof::to_bytes()` as a bincode byte-slice. Bincode prepends a u64 LE length of the inner byte array. Thus the first 8 bytes inside the bincode region are the inner proof length ($= L - 8$), followed by the actual proof bytes.

5.4 R1CSProof Wire Format

After removing the bincode wrapper, the inner proof bytes have this layout:

Offset	Size	Fields
0	$13 \times 32 = 416$	Points: $A_I, A_O, S, T_1^*, T_2, T_3^*, T_4^*, T_5^*, T_6^*, S', T'_1, S'_1, S'_2$.
416	$8 \times 32 = 256$	Scalars: $\hat{t}, \tau_x, \mu, \hat{t}_c, \tau_c, \mu_c, t_\times, \rho_c$.
672	8	u64 LE: IPA sub-proof length ($= \ell_{\text{ipa}}$).
680	8	u64 LE: eCP sub-proof length ($= \ell_{\text{ecp}}$).
688	ℓ_{ipa}	IPA sub-proof.
$688 + \ell_{\text{ipa}}$	ℓ_{ecp}	eCP sub-proof.

Each sub-proof starts with a 96-byte header: k, d, m each stored as a u64 in the first 8 bytes of a 32-byte zero-padded field. Then cross-term points (IPA: singles; eCP: pairs), then base-case scalars. Reject if any point fails decompression or any scalar is $\geq p$.

IPA Sub-Proof Layout

Offset	Size	Fields
0	32	k as u64 LE in bytes 0–7, remaining 24 bytes zero.
32	32	d .
64	32	m .
96	$d \times 2(k-1) \times 32$	Cross-term points $U_j^{(r)}$, round-major.
$96 + \dots$	$m \times 32$	$\mathbf{a}_{\text{fin}}$ (scalars).
$\dots$	$m \times 32$	$\mathbf{b}_{\text{fin}}$ (scalars).

eCP Sub-Proof Layout

Same header structure (k, d, m in 32-byte fields). Then $d \times 2(k-1)$ point *pairs* $(A_{j,0}^{(r)}, A_{j,1}^{(r)})$, each pair being $2 \times 32 = 64$ bytes. Then m scalars for $\mathbf{z}_{\text{fin}}$.

Reject if any compressed point fails decompression or any scalar is non-canonical ($\geq p$).

6 Verification Steps

The verifier (1) reads the public key and ciphertexts from files, (2) derives the challenge vector $\mathbf{e}$, (3) replays the Fiat–Shamir transcript to recover all proof challenges, (4) computes verification scalars for the IPA and eCP sub-proofs, (5) computes the right-hand side by aggregating input ciphertexts, and (6) assembles everything into a single batched MSM. It accepts iff the result equals $\mathcal{O}$.

6.1 Read Inputs and Parse Proof

Read (g, f, h) and $(C_{i,1}, C_{i,2})_{i=1}^n$ from the input file (§5.2). Read $(V, \text{proof}, (C'_{i,1}, C'_{i,2})_{i=1}^n)$ from the output bundle (§5.3). Parse the proof (§5.4). Decompress all points; reject on failure.

Derive the Bulletproof generators $\mathbf{g}, \mathbf{h}$ using the algorithm in §3.2 with capacity $\geq N$ (where N is read from the proof's IPA header).

6.2 Derive Challenge Vector $\mathbf{e}$

Produce a public pseudorandom vector $\mathbf{e} = (e_1, \dots, e_N) \in \mathbb{Z}_p^N$ that binds the challenges to all public inputs. Both prover and verifier compute the same $\mathbf{e}$.

1. Initialize transcript $\mathcal{T}_e$ with label **"FiatShamir-e"**.
2. Absorb(**"dom-sep"**, **"challenge-e-derivation"**).
3. Absorb the public key and a CRS fingerprint:
 - Absorb(**"pk_g"**, $\text{Compress}(g)$)
 - Absorb(**"pk_f"**, $\text{Compress}(f)$)
 - Absorb(**"ck_h"**, $\text{Compress}(h)$)
 - Absorb(**"ck_n"**, N as 8-byte LE)
 - Absorb(**"ck_g0"**, $\text{Compress}(g_1)$) — the first generator from $\mathbf{g}$.
 - Absorb(**"ck_h0"**, $\text{Compress}(h_1)$) — the first generator from $\mathbf{h}$.

Instead of hashing all $2N$ generators (which would be expensive), we hash the capacity N plus the first generator of each chain. Since the chains are deterministic, knowing N , g_1 , and h_1 uniquely identifies the full CRS.

4. Absorb all $4n$ ciphertext components (each as 32-byte compressed point):
 - For $i = 1, \dots, n$: Absorb(**"c1"**, $\text{Compress}(C_{i,1})$)
 - For $i = 1, \dots, n$: Absorb(**"c2"**, $\text{Compress}(C_{i,2})$)
 - For $i = 1, \dots, n$: Absorb(**"c1p"**, $\text{Compress}(C'_{i,1})$)
 - For $i = 1, \dots, n$: Absorb(**"c2p"**, $\text{Compress}(C'_{i,2})$)
5. Squeeze a 32-byte seed: $\sigma \leftarrow \text{challenge_bytes}(\text{"e_seed"}, 32)$.
6. Expand the seed into scalars using a ChaCha20 [7] pseudo-random generator seeded with σ : for $i = 1, \dots, n$, draw 64 bytes, reduce modulo p to get e_i ; for $i = n+1, \dots, N$, set $e_i = 0$.

Output: $\mathbf{e} = (e_1, \dots, e_N) \in \mathbb{Z}_p^N$.

6.3 Compute Aggregated Input Ciphertexts

The verifier needs to check that the output ciphertexts are a re-encryption of a permutation of the input ciphertexts. To this end, the verifier performs two multi-scalar multiplications, each

of width n , representing the “right-hand side” of the shuffle equation:

$$\bar{C}_1 \leftarrow \sum_{i=1}^n e_i \cdot C_{i,1} = \text{MSM}((C_{1,1}, \dots, C_{n,1}), (e_1, \dots, e_n), n),$$

$$\bar{C}_2 \leftarrow \sum_{i=1}^n e_i \cdot C_{i,2} = \text{MSM}((C_{1,2}, \dots, C_{n,2}), (e_1, \dots, e_n), n).$$

6.4 Replay the Proof Transcript

Initialize a new transcript $\mathcal{T}$. Replay the prover’s absorb–squeeze operations in strict order. After this step, the verifier has all challenges $(u, y, z, x, x', x_{\text{agg}}, w, x_{\text{eCP}})$.

Step	Operations
Init	$\mathcal{T} \leftarrow \text{Transcript}::\text{new}(\text{"ShuffleTest"})$ $\text{Absorb}(\text{"dom-sep"}, \text{"ShuffleProof"})$ $\text{Absorb}(\text{"k"}, \text{Scalar}::\text{from}(N) \text{ as 32 bytes})$
R1CS	$\text{Absorb}(\text{"dom-sep"}, \text{"r1cs v1"})$
V	$\text{Absorb}(\text{"V"}, \text{Compress}(V)); \text{Absorb}(\text{"m"}, N \text{ as 8-byte LE})$
u	$u \leftarrow \text{Squeeze}(\text{"k-scalar shuffle challenge"})$
A_I, A_O, S	$\text{Absorb}(\text{"A_I"}, A_I); \text{Absorb}(\text{"A_O"}, A_O); \text{Absorb}(\text{"S"}, S)$
y, z	$y \leftarrow \text{Squeeze}(\text{"y"}); z \leftarrow \text{Squeeze}(\text{"z"})$
T ’s	$\text{Absorb } T_1^* (\text{"T_1"}), T_3^* (\text{"T_3"}), T_4^* (\text{"T_4"}), T_5^* (\text{"T_5"}), T_6^* (\text{"T_6"}),$ then $T_2 (\text{"T_2"})$
x	$x \leftarrow \text{Squeeze}(\text{"x"})$
Eval	$\text{Absorb } \hat{t} (\text{"t_x"}), \tau_x (\text{"t_x_blinding"}), \mu (\text{"e_blinding"})$
Consist	$\text{Absorb } S' (\text{"S_prime"}), T'_1 (\text{"T_1_prime"}), S'_1 (\text{"S1_prime"}), S'_2$ ("S2_prime")
x'	$x' \leftarrow \text{Squeeze}(\text{"x_prime"})$
Consist.	$\text{Absorb } \hat{t}_c (\text{"tc_x"}), \tau_c (\text{"tc_x_blinding"}), \mu_c (\text{"ec_blinding"}), \rho_c$ $(\text{"r_blinding"}), t_\times (\text{"t_cross"})$
$x_{\text{agg}}, w, x_{\text{eCP}}$	$x_{\text{agg}} \leftarrow \text{Squeeze}(\text{"x_ipp"}); w \leftarrow \text{Squeeze}(\text{"w_agg"}); x_{\text{eCP}} \leftarrow$ $\text{Squeeze}(\text{"x_eCP"})$

6.5 Compute Constraint Weight Vectors

Following Algorithm A.1 in Appendix A, this step is a deterministic computation on scalars only.

- $y_{\text{LHS}} = \prod_{i=1}^N (e_i - u)$, a public scalar.
- $\mathbf{w}_L, \mathbf{w}_R$: vectors of length $n-1$, with $w_L[q] = -z^{2q-1}$ and $w_R[q] = -z^{2q}$ for $q = 1, \dots, n-1$.
- $\mathbf{w}_O$: vector of length $n-1$, with $w_O[q] = z^{2q+1}$ for $q = 1, \dots, n-2$ and $w_O[n-1] = z^{2n-1}$.
- $\mathbf{w}_V$: vector of length N , with $w_V[1] = -z$, $w_V[q+1] = -z^{2q}$ for $q = 1, \dots, n-1$, and $w_V[n+j] = -z^{2n+j-1}$ for $j = 1, \dots, N-n$.
- $w_c = u \cdot z + u \cdot \sum_{q=1}^{n-1} z^{2q} + y_{\text{LHS}} \cdot z^{2n-1}$.
- y^{-i} for $i = 1, \dots, N$ (computed iteratively).
- $\delta = \sum_{q=1}^{n-1} w_R[q] \cdot y^{-q} \cdot w_L[q] = \sum_{q=1}^{n-1} z^{4q-1} \cdot y^{-q}$.

6.6 IPA Verification Scalars

The prover used a k -ary folding argument (the IPA) to prove an inner-product relation and sent base-case values $\mathbf{a}_{\text{fin}}, \mathbf{b}_{\text{fin}}$ to the verifier, which is included in the IPA sub-proof (Section 5.4). The verifier “unfolds” these values back to full-size vectors $\mathbf{s}_g, \mathbf{s}_h$. These scalars will be paired with the g_i and h_i generators in the final MSM. Details are provided in Algorithm A.2 in Appendix A.

6.7 eCP Verification Scalars

The eCP (extended ciphertext proof) checks that the committed permutation is consistent with the actual re-encrypted ciphertexts. Like the IPA, it uses k -ary folding. The verifier unfolds the base-case $\mathbf{z}_{\text{fin}}$ into $\mathbf{z}_s$, which will be paired with the output ciphertexts and g -generators in the final MSM. Details are provided in Algorithm A.3 in Appendix A.

6.8 Assemble Scalar Coefficients

Sample $r_b \xleftarrow{\$} \mathbb{Z}_p$ (from transcript-seeded RNG). Define $r_b^2 = r_b \cdot r_b$, $r_b^3 = r_b^2 \cdot r_b$, $r_b^4 = r_b^3 \cdot r_b$.

$$\hat{t}_{\text{agg}} = \hat{t} + x_{\text{agg}} \cdot t_{\times} + x_{\text{agg}}^2 \cdot \hat{t}_c.$$

Fixed-point scalars:

$$\begin{aligned} \alpha_g &= w \cdot s_Q - w \cdot \hat{t}_{\text{agg}} \cdot s_P + r_b(x^2(w_c + \delta) - \hat{t}) - r_b^2 \hat{t}_c + s_{P'} \cdot r_b^3 \cdot \rho_c \\ \alpha_h &= x_{\text{agg}} \cdot \mu_c \cdot s_P + \mu \cdot s_P - r_b^2 \tau_c - r_b \tau_x + s_{P'} \cdot r_b^4 \cdot \mu_c \\ \alpha_f &= s_{P'} \cdot r_b^3 \cdot x_{\text{ecp}} \cdot \rho_c \\ \alpha_V &= -x_{\text{agg}} \cdot s_P + r_b^4 \cdot (-s_{P'}) \\ \alpha_{S'} &= -x_{\text{agg}} \cdot x' \cdot s_P + r_b^4 \cdot x' \cdot (-s_{P'}) \\ \alpha_{A_I} &= -x \cdot s_P, \quad \alpha_{A_O} = -x^2 \cdot s_P, \quad \alpha_S = -x^3 \cdot s_P \\ \alpha_{T_2} &= r_b x^2 + r_b^2 \quad (\text{two equations share } T_2) \\ \alpha_{T'_1} &= r_b^2 \cdot x' \\ \alpha_{S'_1} &= r_b^3 \cdot x' \cdot (-s_{P'}), \quad \alpha_{S'_2} = r_b^3 \cdot x_{\text{ecp}} \cdot x' \cdot (-s_{P'}) \\ \alpha_{\bar{C}_1} &= r_b^3 \cdot (-s_{P'}), \quad \alpha_{\bar{C}_2} = r_b^3 \cdot x_{\text{ecp}} \cdot (-s_{P'}) \end{aligned}$$

T-commitment scalars (for $T_1^*, T_3^*, T_4^*, T_5^*, T_6^*$): $(r_b x, r_b x^3, r_b x^4, r_b x^5, r_b x^6)$.

Generator vector scalars ($i = 1, \dots, N$):

$$\begin{aligned} \alpha_{g_i} &= s_{g,i} - x \cdot w_{R,i} \cdot y^{-i} \cdot s_P + z_{s,i} \cdot r_b^4 \\ \alpha_{h_i} &= y^{-i} \cdot s_{h,i} + (y^{-i}(x \cdot w_{L,i} + w_{O,i}) - 1)(-s_P) - y^{-i} \cdot x_{\text{agg}} \cdot s_P \cdot w_{V,i} \end{aligned}$$

Sub-proof cross-term scalars: IPA: $-s_{U,j}$ for each j . eCP row 0: $-s_{A,j} \cdot r_b^4$. eCP row 1: $-s_{A,j} \cdot r_b^3$.

Output ciphertext scalars ($i = 1, \dots, n$): $\alpha_{C'_{i,1}} = z_{s,i} \cdot r_b^3$, $\alpha_{C'_{i,2}} = z_{s,i} \cdot r_b^3 \cdot x_{\text{ecp}}$.

6.9 Final MSM

Collect all scalar–point pairs and compute a single MSM:

$$\text{result} = \sum_{\text{all pairs}} \alpha_i \cdot P_i$$

using the pairs listed in the following table. The cross-term points from the IPA and eCP sub-proofs are indexed sequentially across all d rounds (round 0 first, then round 1, and so on).

Scalar	Point
$\alpha_{A_I}, \alpha_{A_O}, \alpha_S$	A_I, A_O, S
$\alpha_V, \alpha_{S'}$	V, S'
$\alpha_g, \alpha_h, \alpha_f$	g, h, f
α_{g_i} for $i = 1, \dots, N$	g_i
α_{h_i} for $i = 1, \dots, N$	h_i
$-s_{U,j}$ for $j = 0, \dots, d \cdot 2(k-1) - 1$	IPA cross-term points $U_\ell^{(r)}$, all rounds
$\alpha_{T'_1}$	T'_1
α_{T_2} (combined: $r_b x^2 + r_b^2$)	T_2
$r_b x$	T_1^*
$r_b x^3, r_b x^4, r_b x^5, r_b x^6$	$T_3^*, T_4^*, T_5^*, T_6^*$
$\alpha_{S'_1}, \alpha_{S'_2}$	S'_1, S'_2
$\alpha_{\bar{C}_1}, \alpha_{\bar{C}_2}$	$\bar{C}_1, \bar{C}_2$
$\alpha_{C'_{i,1}}$ for $i = 1, \dots, n$	$C'_{i,1}$
$\alpha_{C'_{i,2}}$ for $i = 1, \dots, n$	$C'_{i,2}$
$-s_{A,j} \cdot r_b^4$ for $j = 0, \dots, d \cdot 2(k-1) - 1$	eCP $A_{\ell,0}^{(r)}$, all rounds
$-s_{A,j} \cdot r_b^3$ for $j = 0, \dots, d \cdot 2(k-1) - 1$	eCP $A_{\ell,1}^{(r)}$, all rounds

ACCEPT if and only if result = $\mathcal{O}$.

A Verification Algorithms

A.1 Algorithm: Compute Weight Vectors

Input: $\mathbf{e} \in \mathbb{Z}_p^N$, $u \in \mathbb{Z}_p$, $z \in \mathbb{Z}_p$, $y \in \mathbb{Z}_p$, n, N .

Output: $\mathbf{w}_L, \mathbf{w}_R, \mathbf{w}_O \in \mathbb{Z}_p^{n-1}$, $\mathbf{w}_V \in \mathbb{Z}_p^N$, $w_c \in \mathbb{Z}_p$, $\delta \in \mathbb{Z}_p$, $(y^{-1}, y^{-2}, \dots, y^{-N})$.

1. Compute $y_{\text{LHS}} \leftarrow \prod_{i=1}^N (e_i - u)$.
2. Initialize $\mathbf{w}_L, \mathbf{w}_R, \mathbf{w}_O \leftarrow \mathbf{0}^{n-1}$; $\mathbf{w}_V \leftarrow \mathbf{0}^N$; $w_c \leftarrow 0$.
3. Set $z_{\text{pow}} \leftarrow z$.
4. **for** $q = 1$ **to** $n - 1$ **do**
 - $w_L[q] \leftarrow w_L[q] - z_{\text{pow}}$ *(left-wire weight for gate q)*
 - **if** $q = 1$:
 - $w_V[1] \leftarrow w_V[1] - z_{\text{pow}} \cdot 1$
 - $w_c \leftarrow w_c + u \cdot z_{\text{pow}}$
 - **else:** $w_O[q-1] \leftarrow w_O[q-1] + z_{\text{pow}}$
 - $z_{\text{pow}} \leftarrow z_{\text{pow}} \cdot z$
 - $w_R[q] \leftarrow w_R[q] - z_{\text{pow}}$ *(right-wire weight for gate q)*
 - $w_V[q+1] \leftarrow w_V[q+1] - z_{\text{pow}} \cdot 1$
 - $w_c \leftarrow w_c + u \cdot z_{\text{pow}}$
 - $z_{\text{pow}} \leftarrow z_{\text{pow}} \cdot z$
- end for**
5. *(Output constraint:)*
 - $w_O[n-1] \leftarrow w_O[n-1] + z_{\text{pow}}$
 - $w_c \leftarrow w_c + y_{\text{LHS}} \cdot z_{\text{pow}}$
 - $z_{\text{pow}} \leftarrow z_{\text{pow}} \cdot z$
6. **for** $j = 1$ **to** $N - n$ **do**
 - $w_V[n+j] \leftarrow w_V[n+j] - z_{\text{pow}}$
 - $z_{\text{pow}} \leftarrow z_{\text{pow}} \cdot z$
- end for**
7. Negate $\mathbf{w}_V$: $w_V[i] \leftarrow -w_V[i]$ for all i .
8. $y^{-1} \leftarrow y^{p-2}$.
for $i = 1$ **to** N : $y^{-i} \leftarrow (y^{-1})^i$ *(iteratively)*.
9. $\delta \leftarrow \sum_{q=1}^{n-1} w_R[q] \cdot y^{-q} \cdot w_L[q]$.

A.2 Algorithm: IPA Verification Scalars

Input: $\{U_j^{(r)}\}$, $\mathbf{a}_{\text{fin}}, \mathbf{b}_{\text{fin}}$, transcript $\mathcal{T}$, N, k, d .

Output: $\mathbf{s}_g, \mathbf{s}_h \in \mathbb{Z}_p^N$, $s_Q, s_P \in \mathbb{Z}_p$, $\mathbf{s}_U \in \mathbb{Z}_p^{d \cdot 2(k-1)}$.

Part A: Derive challenges

1. Absorb IPA domain separator into $\mathcal{T}$:
 - Absorb(b"protocol-name", b"k_bullet_delay")
 - Absorb(b"n", N as 8-byte LE)
 - Absorb(b"k", k as 8-byte LE)
2. **for** $r = 0$ **to** $d - 1$ **do**
 - **for** $j = 0$ **to** $2(k-1) - 1$ **do**
 - Absorb(b"U_round", r as 8-byte LE)
 - Absorb(b"U_index", j as 8-byte LE)
 - Absorb(b"U_point", Compress($U_j^{(r)}$))
 - end for**
 - Absorb(b"challenge_prefix", b"c_")
 - Absorb(b"challenge_index", r as 8-byte LE)
 - $c_r \leftarrow \text{Squeeze}(\text{b"challenge_separator"})$
- end for**
3. Compute c_r^{-1} for each r (batch inversion).

Part B: Compute round lengths

- Set $L_0 = N$.
- **for** $r = 1$ **to** d :
 - $\text{pad} = (k - L_{r-1} \bmod k) \bmod k$
 - $L_r = (L_{r-1} + \text{pad})/k$
- end for.**
- Check $L_d = m$.

Part C: Compute suffix products and scale factor

- Set $\sigma_d = 1$.
- **for** $r = d-1$ **downto** 0 :
 - $\sigma_r = \sigma_{r+1} \cdot c_r^{k-1}$
- end for.**
- Set $s_P = \sigma_0 = \prod_{r=0}^{d-1} c_r^{k-1}$.

Part D: Unfold $\mathbf{a}_{\text{fin}}$ into $\mathbf{s}_g$

- $\mathbf{s}_g \leftarrow \mathbf{a}_{\text{fin}}$ (length m).
- **for** $r = d-1$ **downto** 0 **do**
 - Let $\ell = |\mathbf{s}_g|$. Allocate $\mathbf{s}_g^{\text{new}}$ of length $k \cdot \ell$.
 - **for** $j = 0$ **to** $k-1$ **do**
 - * **for** $i = 0$ **to** $\ell-1$ **do**
 - $\mathbf{s}_g^{\text{new}}[j \cdot \ell + i] \leftarrow \mathbf{s}_g[i] \cdot (c_r^{-1})^j$
 - **end for**
 - **end for**
 - $\mathbf{s}_g \leftarrow \mathbf{s}_g^{\text{new}}[0:L_r]$ (*keep only the first L_r elements*)
- **for** $i = 0$ **to** $N-1$: $\mathbf{s}_g[i] \leftarrow \mathbf{s}_g[i] \cdot s_P$.

Part E: Unfold $\mathbf{b}_{\text{fin}}$ into $\mathbf{s}_h$

Same as Part D but using c_r (not c_r^{-1}) and **not** multiplying by s_P at the end.

Part F: Base-case inner product

$$s_Q = \sum_{i=0}^{m-1} a_{\text{fin},i} \cdot b_{\text{fin},i}.$$

Part G: Cross-term scalars

- **for** $r = 0$ **to** $d-1$ **do**
 - $\sigma = \sigma_{r+1}$ *(suffix product, $\sigma_d = 1$)*
 - **for** $\ell = 1$ **to** $k-1$:
 - * $s_U[\text{idx}] = c_r^{k-1-\ell} \cdot \sigma$; $\text{idx}++$ *(first half: positive cross-terms)*
 - **for** $\ell = 1$ **to** $k-1$:
 - * $s_U[\text{idx}] = c_r^{k-1+\ell} \cdot \sigma$; $\text{idx}++$ *(second half: negative cross-terms)*
 - **end for**

A.3 Algorithm: eCP Verification Scalars

Input: $\{(A_{j,0}^{(r)}, A_{j,1}^{(r)})\}$, $\mathbf{z}_{\text{fin}}$, transcript $\mathcal{T}$, N , k , d .

Output: $\mathbf{z}_s \in \mathbb{Z}_p^N$, $s_{P'} \in \mathbb{Z}_p$, $\mathbf{s}_A \in \mathbb{Z}_p^{d \cdot 2(k-1)}$.

Part A: Derive challenges

Same structure as IPA Part A but with labels: `b"k_ipp_delay_2"`, `b"A_round"`, `b"A_index"`, `b"A_point_0"`, `b"A_point_1"`.

*Note: each cross-term is a **pair** of points $(A_{j,0}^{(r)}, A_{j,1}^{(r)})$, both absorbed per index. Challenges: $c'_0, \dots, c'_{d-1}$.*

Part B: Round lengths

Reuse from IPA.

Part C: Suffix products (exponent k , not $k-1$)

- Set $\sigma'_d = 1$.
- **for** $r = d-1$ **downto** 0:
 - $\sigma'_r = \sigma'_{r+1} \cdot (c'_r)^k$
- end for**.
- $s_{P'} = \sigma'_0 = \prod_{r=0}^{d-1} (c'_r)^k$.

Part D: Unfold $\mathbf{z}_{\text{fin}}$ into $\mathbf{z}_s$

Same as IPA Part D using $(c'_r)^{-1}$ and scaling by $s_{P'}$.

Part E: Cross-term scalars (exponents differ from IPA)

- **for** $r = 0$ **to** $d-1$ **do**
 - $\sigma' = \sigma'_{r+1}$
 - **for** $i = 1$ **to** $k-1$:
 - * $s_A[\text{idx}] = (c'_r)^i \cdot \sigma'$; $\text{idx}++$ *(first half: exponent i)*
 - **for** $i = 1$ **to** $k-1$:
 - * $s_A[\text{idx}] = (c'_r)^{k+i} \cdot \sigma'$; $\text{idx}++$ *(second half: exponent $k+i$)*
- end for**

B Notation Correspondence

Spec notation	Code variable	Description
<i>Sizes and parameters</i>		
n	<code>k_original</code>	Number of real ciphertexts
N	<code>padded_n</code>	Padded count
$n - 1$	<code>n</code> (in <code>VerifierCS</code>)	Number of multiplication gates
k	<code>k_fold / proof.ipp_proof.k</code>	Folding factor
d	<code>proof.ipp_proof.U_vecs.len()</code>	Number of folding rounds
m	<code>proof.ipp_proof.a_final.len()</code>	Base-case vector size
<i>Generators</i>		
g	<code>pc_gens.B</code>	ElGamal / Pedersen value base
h	<code>pc_gens.B_blinding</code>	Pedersen blinding base
f	<code>pc_gens.F</code>	ElGamal public-key base
g_i	<code>bp_gens.G_vec[0][i-1]</code>	Bulletproof g -generators (0-indexed)
h_i	<code>bp_gens.H_vec[0][i-1]</code>	Bulletproof h -generators
<i>Fiat-Shamir challenges</i>		
$\mathbf{e}$	<code>e_vec</code>	Public challenge vector
u	<code>z_shuffle</code>	Shuffle challenge
y	<code>y</code>	Generator-scaling challenge
z	<code>z</code>	Constraint-flattening challenge
x	<code>x</code>	Polynomial evaluation challenge
x'	<code>x_prime</code>	Consistency challenge
x_{agg}	<code>x_ipp</code>	Aggregation challenge
w	<code>w_agg</code>	IPA normalization
x_{ecp}	<code>chall_batched_ecp</code>	eCP row-batching challenge
<i>Proof scalars</i>		
$\hat{t}$	<code>proof.t_x</code>	Evaluated inner product $\langle \mathbf{l}(x), \mathbf{r}(x) \rangle$
τ_x	<code>proof.t_x_blinding</code>	Polynomial blinding at x
μ	<code>proof.e_blinding</code>	Opening blinding $(= \alpha x + \beta x^2 + \rho x^3)$
$\hat{t}_c$	<code>proof.tc_x</code>	Consistency inner product
τ_c	<code>proof.tc_x_blinding</code>	Consistency blinding
μ_c	<code>proof.ec_blinding</code>	Consistency commitment blinding
ρ_c	<code>proof.r_blinding</code>	Re-randomization blinding
t_x	<code>proof.t_cross</code>	Cross-term $\langle \mathbf{l}_c, \mathbf{r} \rangle + \langle \mathbf{l}, \mathbf{r}_c \rangle$
<i>Verification scalars</i>		
r_b	<code>r</code>	Random batching scalar
s_P	<code>s_P_cir</code>	IPA scale factor $\prod c_r^{k-1}$
$s_{P'}$	<code>s_P</code> (eCP return)	eCP scale factor $\prod (c'_r)^k$
s_Q	<code>s_Q_cir</code>	$\langle \mathbf{a}_{\text{fin}}, \mathbf{b}_{\text{fin}} \rangle$
s_g	<code>s_g_cir</code>	IPA g -generator scalars
s_h	<code>s_h_cir</code>	IPA h -generator scalars
s_U	<code>s_U_cir</code>	IPA cross-term scalars
$\mathbf{z}_s$	<code>z_s_vec</code>	eCP expanded witness scalars
$\mathbf{s}_A$	<code>s_A_vec</code>	eCP cross-term scalars
<i>Weight vectors</i>		
$\mathbf{w}_L$	<code>wL</code>	Left-wire weights
$\mathbf{w}_R$	<code>wR</code>	Right-wire weights
$\mathbf{w}_O$	<code>w0</code>	Output-wire weights
$\mathbf{w}_V$	<code>wV / rC</code>	Committed-variable weights
w_c	<code>wc</code>	Constant weight
δ	<code>delta</code>	$\sum w_{R,i} \cdot y^{-i} \cdot w_{L,i}$
y_{LHS}	(computed inline)	$\prod_i (e_i - u)$
<i>Aggregated ciphertexts</i>		
$\bar{C}_1, \bar{C}_2$	<code>C[0], C[1]</code>	MSM of input ciphertexts with $\mathbf{e}$
<i>Proof points</i>		
A_I, A_O, S	<code>proof.A_I</code> , etc.	Circuit wire commitments
$T_1^*, \dots, T_6^*$	<code>proof.T_1, \dots, proof.T_6</code>	Polynomial commitments
S', T_1', S_1', S_2'	<code>proof.S_prime</code> , etc.	Consistency commitments
V	<code>self.V[0]</code>	Permutation commitment
$U_j^{(r)}$	<code>proof.ipp_proof.U_vecs[r][j]</code>	IPA cross-term points
$A_{j,0}^{(r)}, A_{j,1}^{(r)}$	<code>proof.ecp_batched.A_vecs[r][j][0,1]</code>	eCP cross-term pairs
$\mathbf{a}_{\text{fin}}, \mathbf{b}_{\text{fin}}$	<code>proof.ipp_proof.a_final, .b_final</code>	IPA base-case vectors
$\mathbf{z}_{\text{fin}}$	<code>proof.ecp_batched.z</code>	eCP base-case vector

References

- [1] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, “Bulletproofs: Short proofs for confidential transactions and more,” *IEEE S&P*, 2018.
- [2] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” *CRYPTO*, 1986.
- [3] H. de Valence, J. Grigg, G. Tankersley, F. Valsorda, and I. Lovecruft, “The Ristretto group,” <https://ristretto.group>, 2018.
- [4] H. de Valence, J. Grigg, M. Hamburg, I. Lovecruft, G. Tankersley, and F. Valsorda, “The Ristretto255 and Decaf448 Groups,” RFC 9496, 2023.
- [5] A. Faz-Hernandez, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood, “Hashing to Elliptic Curves,” RFC 9380, 2023.
- [6] NIST, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, 2015.
- [7] D. J. Bernstein, “ChaCha, a variant of Salsa20,” 2008. See also RFC 8439.
- [8] H. de Valence, “Merlin: composable proof transcripts for public-coin arguments of knowledge,” <https://merlin.cool>, 2018.
- [9] M. Hamburg, “The STROBE protocol framework,” <https://strobe.sourceforge.io>, 2017.